Lectura: Conceptos básicos de JavaScript

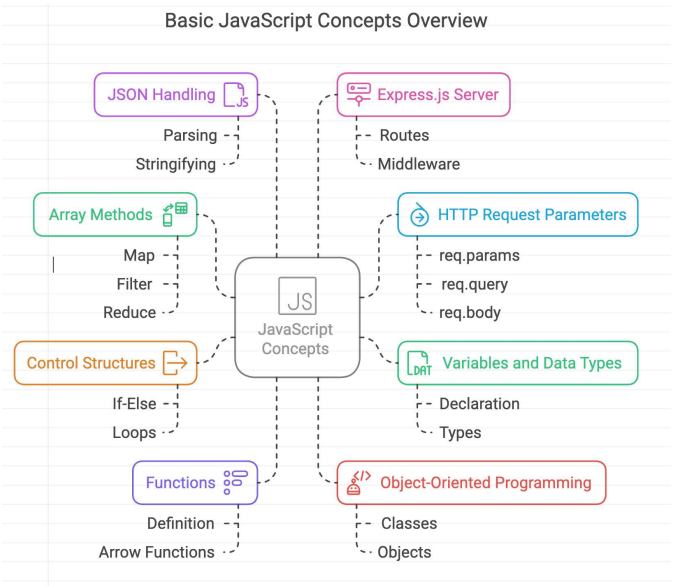
Tiempo estimado: 30 minutos

Objetivos de aprendizaje

- Describir la declaración de variables y los tipos de datos en JavaScript
- Aplicar estructuras de control para el flujo lógico
- Definir y usar funciones, incluidas las funciones de flecha
- Explicar el funcionamiento de los métodos de arreglo (`map`, `filter`, `reduce`) para la transformación de datos
- Implementar programación orientada a objetos básica con clases
- Describir cómo analizar y convertir datos JSON
- Demostrar cómo configurar un servidor Express.js simple y definir rutas básicas
- Explicar cómo usar `req.params`, `req.query` y `req.body` en Express

Requisitos previos

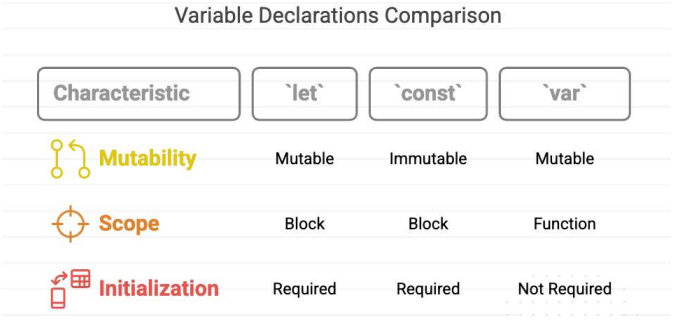
Como se destaca en la Descripción del curso, se requiere un conocimiento básico de JavaScript para este curso. Esta lectura incluye conceptos adicionales de JavaScript para apoyar y mejorar su comprensión.



Variables y tipos de datos

JavaScript utiliza `let`, `const` y `var` para declarar variables.

```
let name = "Book Review";  // string
const rating = 4.5;        // number (immutable binding)
var isPublished = true;    // boolean (function-scoped)
let book = { title: "JS Guide", author: "MDN" }; // object
let tags = ["JavaScript", "Express"];            // array
```



Estructuras de control

Las estructuras de control controlan el flujo lógico en los programas.

```
// Conditional
if (rating > 4) {
  console.log("Highly rated!");  // executes if condition is true
```

```
} else {
  console.log("Needs improvement.");  // executes otherwise
}
// Loop
for (let tag of tags) {
  console.log(tag);  // prints each tag
}
```

Funciones

Las funciones encapsulan la lógica del código para su reutilización.

```
// Function Declaration
function greet(user) {
  return `Hello, ${user}`;  // returns a greeting
}
// Arrow Function
const add = (a, b) => a + b;  // concise syntax for function
console.log(greet("Dev")); // Hello, Dev
```

Métodos de Array

Utilizados para procesar colecciones de datos.

```
const books = [
  { title: "JS Basics", rating: 5 },
  { title: "Node Intro", rating: 3 }
];
// Filter: Get books with rating > 4
const topBooks = books.filter(book => book.rating > 4);
// Map: Get an array of book titles
const titles = books.map(book => book.title);
// Reduce: Calculate average rating
const averageRating = books.reduce((sum, book) => sum + book.rating, 0) / books.length;
console.log(topBooks);      // [{ title: "JS Basics", rating: 5 }]
console.log(titles);        // ["JS Basics", "Node Intro"]
console.log(averageRating); // 4
```

JavaScript orientado a objetos

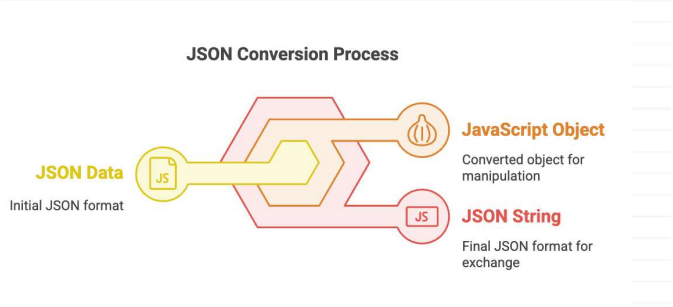
Organiza el código utilizando clases y objetos.

```
class Book {
  constructor(title, rating) {
    this.title = title;
    this.rating = rating;
  }
  describe() {
    return `${this.title} has a rating of ${this.rating}`;
  }
}
const b1 = new Book("Node Basics", 4.2);
console.log(b1.describe());
```

Trabajando con JSON

Esencial para las API: los datos se intercambian en formato JSON.

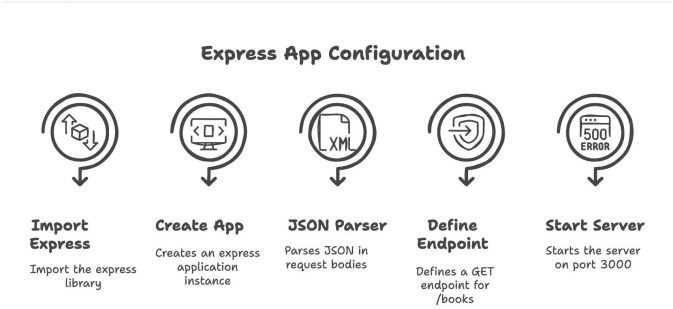
```
let jsonData = '{"title":"Express 101","rating":5}';
const bookObj = JSON.parse(jsonData);    // convert JSON to JS object
const newJson = JSON.stringify(bookObj); // convert JS object to JSON
```



Ejemplo simple de Node.js + Express

Crea un servidor mínimo que responde con datos JSON.

```
// app.js
const express = require("express");
const app = express();
app.use(express.json());  // to parse JSON bodies
app.get("/books", (req, res) => {
  res.json([{ title: "Learn Node", rating: 4 }]);  // return JSON response
});
app.listen(3000, () => console.log("Server running on port 3000"));
```



Resumen de métodos HTTP (API REST)

Utilizados para operaciones CRUD en servicios web.

Método	Propósito
GET	Leer datos
POST	Crear datos
PUT	Actualizar datos
DELETE	Eliminar datos

Entendiendo `req.params`, `req.query` y `req.body`

Estas son formas de acceder a los datos de la solicitud entrante en Express.

1. `req.params`

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;  // extract dynamic part from URL
  res.send(`User ID is ${userId}`);
});
```

Visitar `/users/123` establece `req.params.id` en `"123"`

2. `req.query`

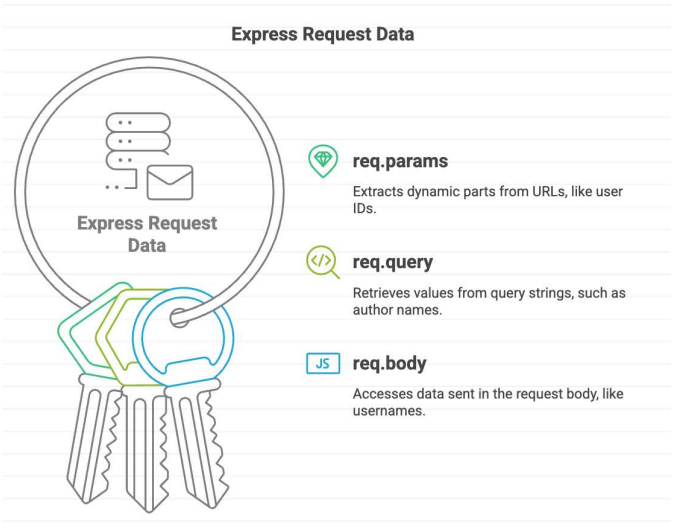
```
app.get('/books', (req, res) => {
  const author = req.query.author;  // extract ?author= value from query string
  res.send(`Filter by author: ${author}`);
});
```

Visitar `/books?author=JohnDoe` establece `req.query.author` a `"JohnDoe"`

3. `req.body`

```
app.post('/register', (req, res) => {
  const username = req.body.username;  // get value from request body
  res.send(`Username received: ${username}`);
});
```

Para JSON `{ "username": "aname", "password": "pwd123" }`, `req.body.username` devuelve `"aname"`



Resumen

En esta lectura, exploraste los conceptos fundamentales de JavaScript esenciales para el desarrollo del lado del servidor con Node.js y Express. Estos incluyeron variables, funciones, métodos de arreglos, clases y trabajo con JSON. También adquiriste una comprensión de cómo opera un servidor básico de Express y cómo manejar los datos de las solicitudes entrantes utilizando req.params, req.query y req.body.

Autor(es)

Ritika Joshi

